

Temeraire: Redis-Reproduktion mit öffentlichem Code

Von der hugepage-aware Idee bis zum funktionierenden Experiment

Aldo Costa

Heinrich-Heine-Universität Düsseldorf

Juli 2026

Was wird hier reproduziert?

Temeraire

Hugepage-bewusster Backend-Allocator für
TCMalloc

Hunter et al., OSDI 2021

- Ziel im Paper: weniger TLB-Druck durch bessere Platzierung
- Ziel hier: der öffentliche Redis 6.0.9-Case-Study-Teil
- Kein Google-Fleet-Experiment, keine Produktions-Telemetrie

Arbeitsfrage

Kann die Redis-Methode mit öffentlichem Code und lokaler Hardware so nachgebaut werden, dass die kleine Effektgröße des Papers sichtbar wird?

Hugepages als Ausgangspunkt

- Normale Seiten: 4 KiB
- Hugepages: 2 MiB, also 512 normale Seiten
- Ein Hugepage-TLB-Eintrag deckt viel mehr Speicher ab
- Der Gewinn ist weniger Adressübersetzung, nicht ein größerer Cache

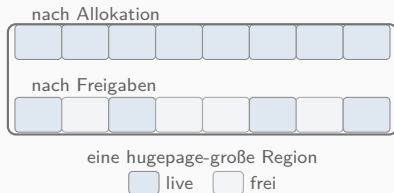
Problem

Hugepages helfen nur, wenn der Speicher passend ausgerichtet und zusammenhängend bleibt.

Allocator-Rolle

`malloc` entscheidet indirekt, ob live Objekte eine Hugepage erhalten, fragmentieren oder für die Freigabe blockieren.

Fragmentierung innerhalb einer Hugepage

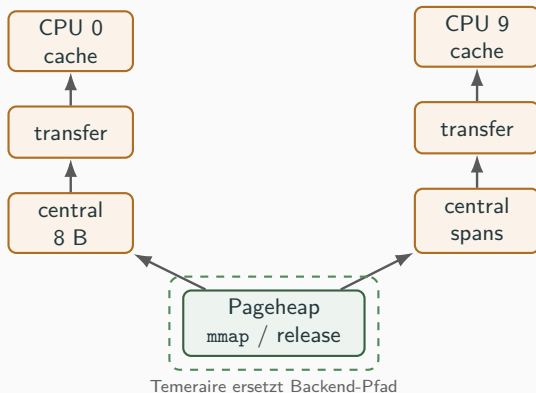


- Freier Speicher reicht nicht aus, wenn er zwischen live Objekten liegt
- Teilfreigabe kann RAM sparen, aber Hugepage-Abdeckung zerstören
- Nichts freigeben kann TLB-Abdeckung halten, kostet aber Speicher

Temeraire macht diesen Trade-off explizit.

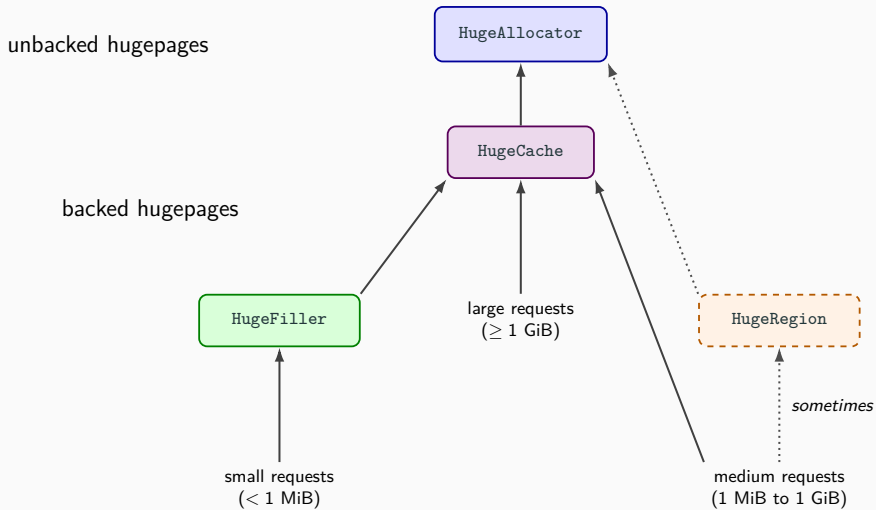
Es packt kleine Objekte bewusst auf schon teilgefüllte Hugepages.

Wo Temeraire in TCMalloc eingreift



- Obere Caches bleiben TCMalloc
- Temeraire ersetzt den Backend-Pageheap
- Reproduktion vergleicht deshalb Legacy Pageheap gegen den hugepage-aware Pfad

Temeraire Backend



Redis relevant: Der Listen-Workload trifft vor allem den kleinen Pfad über **HugeFiller**.

Was die Backend-Komponenten machen

HugeAllocator

holt hugepage-ausgerichtete Bereiche vom System und gibt sie an den Backend-Pfad weiter

HugeCache

hält komplett leere, bereits backed Hugepages bereit, damit sie wiederverwendet werden können

HugeFiller

nimmt kleine Allokationen und packt sie auf passende, schon teilweise belegte Hugepages

HugeRegion

behandelt mittlere Allokationen, bei denen ein normaler Filler-Pfad zu viel freien Rest blockieren würde

Redis landet in diesem Experiment vor allem bei HugeFiller: viele kleine Listen- und String-Allokationen, keine großen zusammenhängenden Blöcke.

Workload aus dem Paper

- Redis 6.0.9
- LPUSH von fünf Werten
- LRANGE 0 4
- 2000 Trials, je 1 Mio. Requests

Erwarteter Effekt

Redis erzeugt viele kleine Listen- und String-Allokationen. Wenn `HugeFiller` sie besser platziert, sinkt der TLB-Druck.

$$\text{kombiniert} = \frac{2}{1/r_{\text{LPUSH}} + 1/r_{\text{LRANGE}}}$$

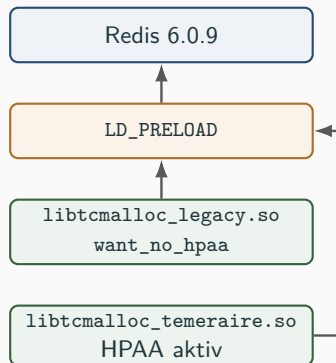
Konsequenz für die Messung

Der erwartete Unterschied liegt unter einem Prozent. Reihenfolge, THP-Zustand und Laufzeit-Validierung dürfen nicht implizit bleiben.

Rolle der Wrapper

Redis wird nicht zweimal umgebaut. Stattdessen werden zwei kleine TCMalloc-Shared-Libraries gebaut und beim Start in den Prozess geladen.

- gleicher Redis-Build
- gleiche Wrapper-Quelle
- unterschiedliche TCMalloc-Backend-Konfiguration



Abgrenzung

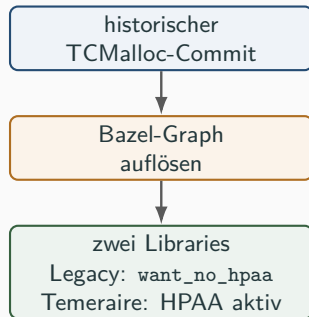
Der Wrapper ist kein neuer Allocator. Er macht den historischen TCMalloc-Code preloadbar, damit der Legacy-vs.-Temeraire-Pfad messbar wird.

Problem

Der Vergleich sollte nicht an selbst nachgebauten Link-Flags oder unvollständigen TCMalloc-Abhängigkeiten hängen.

- TCMalloc ist selbst ein Bazel-Projekt
- Abseil und interne Targets kommen aus dem nativen Build-Graph
- `want_no_hpaa` ist ein TCMalloc-Target, kein Runtime-Flag

Bazel brachte TCMallocs Build-Graphen mit: Abseil, interne Targets und Link-Semantik mussten nicht in CMake nachgebaut werden.



Codepfad: Wrapper und Bazel

Bazel-Target für Legacy

```
cc_binary(  
    name = "temeraire_libtcmalloc_legacy.so",  
    srcs = ["temeraire_wrapper.cc"],  
    linkshared = 1,  
    malloc = "//tcmalloc",  
    deps = ["//tcmalloc:want_no_hpa"],  
)
```

Temeraire-Target

```
name = "temeraire_libtcmalloc_temeraire.so"  
malloc = "//tcmalloc"
```

Auswahl beim Redis-Start

```
LD_PRELOAD="$GOOGLE_TCMALLOC_LEGACY_LIB"  
LD_PRELOAD="$GOOGLE_TCMALLOC_TEMERAIRE_LIB"
```

Startpunkt: falsche Nähe zum Paper

Vorher

- Redis 7.2.5 statt Redis 6.0.9
- glibc gegen gperftools
- kein Legacy-Pageheap gegen Temeraire
- Scope war anfangs zu breit formuliert

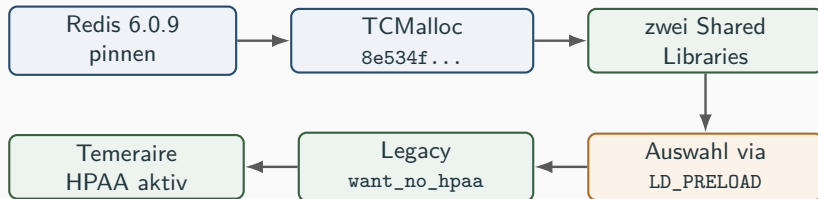
Entscheidung

Das Artefakt wird auf den Redis-Case-Study-Teil eingegrenzt. Die Fleet- und Rollout-Ergebnisse bleiben außerhalb der Reproduktion.

Erster Fix

Erst die Behauptung begrenzen, dann die Technik bauen.

Umbau 1: Den richtigen Vergleich bauen



- Der historische Commit enthält noch den Schalter für den Legacy-Pfad
- Beide Varianten kommen aus derselben Wrapper-Quelle
- Die intendierte Differenz ist der TCMalloc-Backend-Pfad

Was nicht sauber funktionierte

- Moderner TCMalloc-Checkout passte nicht zur Wrapper-Strategie
- Externer Bazel-Wrapper erbte nicht alle Dependencies
- `want_no_hpaa` war nicht global sichtbar

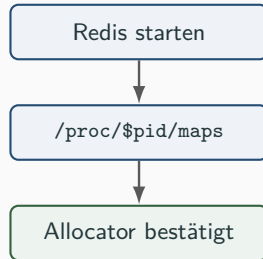
Was am Ende funktionierte

- Wrapper in `tcmalloc/testing`
- Bazelisk und Bazel-Version pinnen
- LLVM-Pfad durch Make, Autotools und Bazel reichen
- fehlende Release-Symbole dynamisch auflösen

- Redis kann weiter `mem_allocator: libc` melden
- Diese Meldung kommt aus der Redis-Build-Konfiguration
- Für `LD_PRELOAD` zählt, was der Prozess wirklich mappt

Prüfung

`/proc/$pid/maps` muss die passende Library zeigen:
`libtcmalloc_legacy.so` oder
`libtcmalloc_temeraire.so`.



Fehlerbild: THP war nicht aktiv genug

Beobachtung in frühen Runs

`AnonHugePages: 0 kB`

Redis lief, die Allocator-Varianten liefen, aber der Hugepage-Mechanismus war nicht belegt.

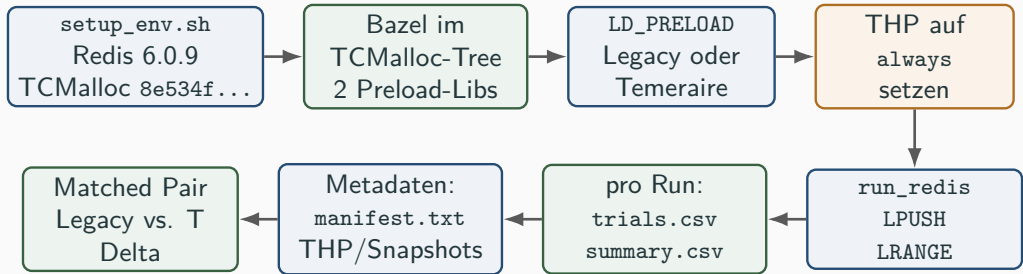
Fix im Harness

- `thp-before.txt` und `thp-after.txt`
- `memory-sample-0250..2000.txt`
- `smaps_rollup` mit `AnonHugePages`
- nur THP-aktive Runs für das Paper-Signal verwenden

Kriterium für gültige Runs

Spätere Runs liefen mit `[always] madvise never`; Redis-Snapshots zeigten 14–18 MiB `AnonHugePages` im Prozess. Erst dann war der Hugepage-Pfad wirklich Teil der Messung.

Ablauf des gültigen Runs



Was technisch passiert

- ein Redis-Binary, zwei TCMalloc-.sos
- Legacy: want_no_hpa; T: HPAA aktiv
- LD_PRELOAD wählt pro Redis-Prozess

Was als Ergebnis bleibt

- summary.csv: LPUSH/LRANGE-Mittel
- Snapshots: THP und AnonHugePages
- Delta nur gegen passenden Legacy-Run

Konfiguration

- 2000 Trials pro Modus
- 1 Mio. Requests pro Trial
- 50 Clients, Pipeline 16
- Redis 6.0.9, TCMalloc 8e534f...

Auswertung

- `summary.csv`: Mittelwert für LPUSH und LRANGE
- `trials.csv`: alle Einzelmessungen bleiben erhalten
- harmonisches Mittel als kombinierte Rate
- Temeraire-Delta immer gegen passenden Legacy-Run

Lesart

Bei Effekten unter einem Prozent ist ein einzelner lokaler Run kein Urteil. Die Wiederholungen zeigen eher ein Muster als eine exakte Zahl.

Release-on war faktisch nicht eingeschaltet

Was der Runner tat

- Background-Thread gestartet
- Release-Rate nicht gesetzt
- TCMalloc-Default im gepinnten Commit: 0 B/s

Folge

Der Thread pflegte weiterhin Per-CPU-Caches, gab aber keine Pageheap-Bytes periodisch mit `ReleaseMemoryToSystem` frei.

Was das f"ur die Auswertung bedeutet

Die bisherigen Release-on-Werte beschreiben eine falsch konfigurierte Bedingung. Sie sind kein Vergleich mit dem +0.44%-Wert aus dem Paper. F"ur einen neuen Lauf braucht es eine explizite positive Rate; das Paper nennt die verwendete Rate nicht.

Run	Reihenfolge	Delta
THP fixed-order	L first	+1.88%
balanced 1	L first	+0.43%
balanced 2	T first	+0.48%
balanced 3	L first	-0.76%
balanced 4	T first	+3.46%

Befund

Vier von fünf THP-aktiven vollständigen Runs favorisieren Temeraire. Der Median liegt bei +0.48%.

Paper-Vergleich

Redis[†] im Paper: +0.75%
Periodic release disabled.

Release-on: nur Diagnosematerial

Run	Reihenfolge	Delta
THP fixed-order	L first	+0.26%
balanced 1	L first	+0.12%
balanced 2	T first	+0.38%
balanced 3	L first	-2.34%
balanced 4	T first	-12.02%
targeted rerun	T first	+0.12%

Kein Paper-Vergleich

Alle Werte entstanden bei einer effektiven Pageheap-Release-Rate von 0 B/s. Der Name `release-on` im Manifest war damit irreführend.

Was von den Runs bleibt

Die Logs zeigen den Konfigurationsfehler und die lokale Streuung. Die Deltas werden nicht mehr als Release-on-Ergebnis interpretiert.

Rerun-Rohwerte: T LPUSH=786686, LRANGE=1195121; L LPUSH=787017, LRANGE=1190716.

Auswertbarer Befund

Release-off liegt mit +0.48% Median im kleinen Effektbereich des Paper-Werts von +0.75%. Die fünf Run-Paare streuen jedoch deutlich.

Offener Befund

Release-on ist noch nicht gemessen. Die vorhandenen Runs hatten eine Pageheap-Release-Rate von 0 B/s und werden neu aufgesetzt.

Vertretbare Aussage

Das Artefakt rekonstruiert den Redis-Vergleich mit öffentlichem Code. Release-off liefert ein kleines, aber stark streuendes Signal; Release-on ist offen.

Nicht reproduziert

- Google-Fleet-Experiment
- Produktions-Rollout
- interne Allocator-Binaries
- exakte Paper-Hardware

Lokale Abweichungen

- Docker/WSL2 und geteilter Kernel
- Consumer i7 statt Skylake Xeon Server
- THP-Policy aus der Host/VM-Umgebung
- Public TCMalloc als historische Annäherung

Nutzen der Grenzen

Sie trennen den methodisch reproduzierten Redis-Pfad von den Teilen des Papers, die ohne Google-Infrastruktur nicht nachstellbar sind.

Technische Lektion

Der schwere Teil war nicht `redis-benchmark`, sondern der Weg zu einem echten Legacy-vs.-Temeraire-Vergleich mit aktiven Hugepages.

- Vergleich aus public TCMalloc rekonstruiert
- Preload zur Laufzeit validiert
- THP-Fehlerbild gefunden und instrumentiert
- Release-on-Konfigurationsfehler im Nachaudit gefunden

Endstand

Release-off zeigt ein kleines positives Mediansignal bei hoher Streuung. Release-on muss mit einer expliziten positiven Rate neu gemessen werden.